10. What is Pandas Library?

The Pandas library is a popular open-source data manipulation and analysis tool for the Python programming language. It provides easy-to-use data structures and data analysis tools for handling and analysing structured data. Pandas is built on top of the NumPy library, which provides efficient numerical operations in Python.

The main data structures in Pandas are the Data Frame and the Series. A Data Frame is a two-dimensional table-like data structure, similar to a spreadsheet or a SQL table, where data is organized in rows and columns. A Series, on the other hand, is a one-dimensional labelled array that can hold any data type.

Pandas provides a wide range of functions and methods to manipulate and transform data, including merging, reshaping, slicing, filtering, and aggregating data. It also offers powerful tools for handling missing data, handling time series data, and reading/writing data in various formats such as CSV, Excel, SQL databases, and more.

Some key features of Pandas include:

1. Data alignment and handling missing data: Pandas provides tools to align and combine data from different sources, handling missing values effectively.

2. Data filtering and selection: You can select, filter, and manipulate data using intuitive indexing techniques, such as label-based indexing, Boolean indexing, and integer-based indexing.

3. Data transformation: Pandas allows you to reshape, pivot, and transform data to suit your needs. It provides functions for sorting, merging, and joining data sets.

4. Time series analysis: Pandas has built-in functionality for handling time series data, including date/time indexing, resampling, and frequency conversion.

5. Data visualization: While Pandas itself doesn't offer visualization capabilities, it integrates well with other libraries like Matplotlib and Seaborn, allowing you to create informative plots and charts to visualize your data.

Overall, Pandas is widely used in data analysis, scientific research, finance, and many other domains where working with structured data is essential. Its simplicity, flexibility, and powerful tools make it a valuable library for data manipulation and analysis tasks in Python.

11. What is Padas Series? Give example.

A Pandas Series is a one-dimensional labelled array that can hold data of any type (integer, float, string, etc.). It is similar to a column in a spreadsheet or a one-dimensional array in NumPy. The Series provides powerful indexing capabilities and functions for data manipulation and analysis.

```
import pandas as pd
# Creating a Series from a list
data = [10, 20, 30, 40, 50]
series = pd.Series(data)
print(series)
```

output:

```
0    10
1    20
2    30
3    40
4    50
dtype: int64
```

In this example, we first import the Pandas library using import pandas as pd. Then, we create a list called data with five elements. We pass this list to the pd.Series() function to create a Series object called series.

When we print the series, it displays the index on the left side (0 to 4) and the corresponding values (10, 20, 30, 40, 50). The dtype: int64 indicates that the data type of the Series is 64-bit integer.

Series objects have both a value and an index. By default, the index is automatically assigned as a sequence of integers starting from 0.

12. Write Python code to create Data frame from a dictionary and display its contents.

```
import pandas as pd
# Create a dictionary
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, 30, 35, 40],
    'City': ['New York', 'London', 'Paris', 'Tokyo']
}
# Create a DataFrame from the dictionary
df = pd.DataFrame(data)
# Display the DataFrame
print(df)
```

OUTPUT:

```
      Name Age      City
0    Alice   25 New York
1      Bob   30    London
2      Charlie   35     Paris
3    David   40     Tokyo
```

In this example, we first import the Pandas library using import pandas as pd. Then, we create a dictionary called data with three keys ('Name', 'Age', 'City') and their corresponding values.

We pass this dictionary to the pd.DataFrame() function to create a DataFrame object called df. The keys of the dictionary ('Name', 'Age', 'City') become the column names of the DataFrame, and the values become the data in each column.

Finally, we use the print() function to display the contents of the DataFrame. Each column is labeled with its corresponding column name, and the rows are indexed starting from 0.

You can access and manipulate the data in the DataFrame using various DataFrame methods and attributes provided by Pandas.

13. Write Python code to create Dataframe from a tuple and display its contents.

```
import pandas as pd
# Create a tuple
data = [
    ('Alice', 25, 'New York'),
    ('Bob', 30, 'London'),
    ('Charlie', 35, 'Paris'),
    ('David', 40, 'Tokyo')
]
# Create a DataFrame from the tuple
df = pd.DataFrame(data, columns=['Name', 'Age', 'City'])
# Display the DataFrame
print(df)
```

OUTPUT:

```
      Name  Age      City
0    Alice   25  New York
1      Bob   30    London
2  Charlie   35     Paris
3    David   40     Tokyo
```

In this code, we start by importing the Pandas library using import pandas as pd. Then, we create a tuple called data, where each element of the tuple represents a row in the DataFrame.

We pass this tuple to the pd.DataFrame() function to create a DataFrame object called df. Additionally, we specify the column names using the columns parameter, which is set to ['Name', 'Age', 'City'].

Finally, we use the print() function to display the contents of the DataFrame. Each column is labeled with its corresponding column name, and the rows are indexed starting from 0.

14. What is Pandas DataFame ? How it is created?

A Pandas DataFrame is a two-dimensional labeled data structure, similar to a table or a spreadsheet. It consists of rows and columns, where each column can contain data of different types (e.g., integers, floats, strings) and each row represents a separate observation or record.

A DataFrame is one of the core data structures provided by the Pandas library, and it offers powerful tools for data manipulation, analysis, and exploration.

To create a Pandas DataFrame, you have several options:

1. From a dictionary: You can create a DataFrame from a dictionary where the keys represent column names, and the values represent the data in each column.

```
import pandas as pd
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['New York', 'London', 'Paris']
}
df = pd.DataFrame(data)
```

2. From a NumPy array: You can create a DataFrame from a NumPy array, providing the data and specifying the column names.

```
import pandas as pd
import numpy as np
data = np.array([
    ['Alice', 25, 'New York'],
    ['Bob', 30, 'London'],
    ['Charlie', 35, 'Paris']
])

df = pd.DataFrame(data, columns=['Name', 'Age', 'City'])
```

3. From a CSV file: You can read data from a CSV file using the pd.read_csv() function and create a DataFrame.

```
import pandas as pd

df = pd.read_csv('data.csv')
```

4. From other data sources: Pandas also provides functions to create DataFrames from various other data sources, such as Excel files, SQL databases, JSON, and more.

16. How to add new column to daataframe ?

To add a new column to a Pandas DataFrame, you can assign a new list or array of values to a new column name. Here's an example:

```
import pandas as pd
# Create a DataFrame
data = {
   'Name': ['Alice', 'Bob', 'Charlie'],
   'Age': [25, 30, 35],
   'City': ['New York', 'London', 'Paris']
}
df = pd.DataFrame(data)
# Add a new column
df['Salary'] = [50000, 60000, 70000]
# Display the DataFrame
print(df)
```

In this example, we start by creating a DataFrame using the provided dictionary data. The DataFrame df has columns 'Name', 'Age', and 'City'.

To add a new column, we use the bracket notation ([]) with the new column name, 'Salary', and assign a list of values [50000, 60000, 70000] to it. The length of the list must match the number of rows in the DataFrame.

17. Give the Python code to create datafram from Excel file.

To create a DataFrame from an Excel file in Python using the Pandas library, you can use the pd.read_excel() function. Here's an example code:

```
import pandas as pd

# Read Excel file and create DataFrame
df = pd.read_excel('data.xlsx', sheet_name='Sheet1')

# Display the DataFrame
print(df)
```

In this code, we import the Pandas library using import pandas as pd. Then, we use the pd.read_excel() function to read the Excel file named 'data.xlsx' and create a DataFrame object. The sheet_name parameter specifies the name of the sheet within the Excel file from which you want to read the data. In this example, we assume the sheet name is 'Sheet1'.

18. Give Python code to find maximum and minimum values for particular column of dataframe.

To find the maximum and minimum values for a particular column in a Pandas DataFrame, you can use the `max()` and `min()` functions on the desired column. Here's an example:

```
import pandas as pd
# Create a DataFrame
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'Salary': [50000, 60000, 70000]
}

df = pd.DataFrame(data)

# Find the maximum and minimum values in the 'Salary' column
max_salary = df['Salary'].max()
min_salary = df['Salary'].min()

# Display the results
print("Maximum Salary:", max_salary)
print("Minimum Salary:", min_salary)
```

Output:

```
Maximum Salary: 70000
Minimum Salary: 50000
```

In this example, we create a DataFrame `df` with columns 'Name', 'Age', and 'Salary'. We then use the `max()` and `min()` functions on the 'Salary' column of the DataFrame to find the maximum and minimum values, respectively.

The `df['Salary']` syntax accesses the 'Salary' column of the DataFrame, and calling `.max()` and `.min()` on it returns the maximum and minimum values, respectively.

Finally, we display the maximum and minimum salary values using the `print()` function.

19. What is Data Visualization ?

Data visualization is the graphical representation of information and data. By using visual elements like charts, graphs, and maps, data visualization tools provide an accessible way to see and understand trends, outliers, and patterns in data.

The importance of data visualization is simple: it helps people see, interact with, and better understand data. Whether simple or complex, the right visualization can bring everyone on the same page, regardless of their level of expertise.

20. What is matplotlib and pyplot.

Pyplot is an API (Application Programming Interface) for Python's matplotlib that effectively makes matplotlib a viable open-source alternative to MATLAB.

Matplotlib is a library for data visualization, typically in the form of plots, graphs and charts.

21. Explain basic arithmetic operations on NumPy array with examples.

NumPy is a powerful library for numerical computations in Python. It provides support for performing various arithmetic operations on arrays efficiently. Here are some basic arithmetic operations that can be performed on NumPy arrays, along with examples:

1. Addition:
The addition operation adds corresponding elements of two arrays.

```python
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

result = a + b
print(result)
```

Output:
```
```

```
[5 7 9]
```

2. Subtraction:
The subtraction operation subtracts corresponding elements of two arrays.

```
import numpy as np

a = np.array([4, 5, 6])
b = np.array([1, 2, 3])

result = a - b
print(result)
```

Output:

```
[3 3 3]
```

3. Multiplication:
The multiplication operation multiplies corresponding elements of two arrays.

```python
import numpy as np

a = np.array([2, 3, 4])
b = np.array([5, 6, 7])

result = a * b
print(result)
```

Output:

```
[10 18 28]
```

4. Division:
The division operation divides corresponding elements of two arrays.

```python
import numpy as np

a = np.array([10, 12, 14])
```

```
b = np.array([2, 3, 4])

result = a / b
print(result)
```

Output:

```
[5. 4. 3.5]
```

5. Exponentiation:

The exponentiation operation raises each element of an array to a specified power.

```python
import numpy as np

a = np.array([2, 3, 4])

result = np.power(a, 3)
print(result)
```

Output:

```
[ 8 27 64]
```

These are just a few examples of basic arithmetic operations that can be performed on NumPy arrays. NumPy provides many more mathematical functions and operations that enable complex mathematical computations and array manipulations.

22. With code examples explain creating pandas' series using Scalar data and Dictionary.

1. Creating a Pandas Series using Scalar data:

You can create a Pandas Series using scalar data by specifying the data value and, optionally, an index label. Here's an example:

```
import pandas as pd

# Create a Series with scalar data
```

```
s = pd.Series(5)
print(s)
```

Output:

```
0    5
dtype: int64
```

In this example, we use the `pd.Series()` constructor and pass the scalar value `5` as the data. By default, the Series is assigned an index label of `0`. The resulting Series contains a single element with the value `5`.

2. Creating a Pandas Series using a Dictionary:

You can create a Pandas Series using a dictionary, where the dictionary keys become the index labels and the values become the data. Here's an example:

```
import pandas as pd
# Create a Series from a dictionary
data = {'A': 10, 'B': 20, 'C': 30}
s = pd.Series(data)
print(s)
```

Output:

```
A    10
B    20
C    30
dtype: int64
```

In this example, we provide a dictionary `data` with three key-value pairs. The dictionary keys 'A', 'B', and 'C' become the index labels of the resulting Series, and the corresponding values `10`, `20`, and `30` become the data in the Series.

By default, the Series uses the dictionary keys as the index labels, and the data type of the Series is inferred based on the data provided (in this case, integers).

22. Explain any four string processing methods supported by Pandas Library with example.

Pandas provides various string processing methods that can be applied to string columns in a DataFrame. Here are four commonly used string processing methods supported by the Pandas library, along with examples:

1. `str.upper()` and `str.lower()`:

These methods convert the strings in a column to uppercase or lowercase, respectively.

```
import pandas as pd

# Create a DataFrame
data = {'Name': ['Alice', 'Bob', 'Charlie']}
df = pd.DataFrame(data)


# Convert the 'Name' column to uppercase
df['Name_upper'] = df['Name'].str.upper()
# Convert the 'Name' column to lowercase
df['Name_lower'] = df['Name'].str.lower()
print(df)
```

Output:

```
    Name Name_upper Name_lower
0   Alice      ALICE      alice
1     Bob        BOB        bob
2  Charlie   CHARLIE    charlie
```

In this example, the `str.upper()` method is used to convert the 'Name' column to uppercase, and the result is assigned to a new column 'Name_upper'. Similarly, the `str.lower()` method is used to convert the 'Name' column to lowercase, and the result is assigned to a new column 'Name_lower'.

2. `str.contains()`:

This method checks if each string in a column contains a specified substring and returns a Boolean Series.

```
import pandas as pd

# Create a DataFrame

data = {'Name': ['Alice', 'Bob', 'Charlie']}

df = pd.DataFrame(data)

# Check if 'Name' column contains the substring 'li'

df['Contains_li'] = df['Name'].str.contains('li')

print(df)
```

Output:

```
    Name  Contains_li
0   Alice         True
1     Bob        False
2  Charlie         True
```

In this example, the `str.contains()` method is used to check if each string in the 'Name' column contains the substring 'li'. The result is assigned to a new column 'Contains_li', which contains Boolean values indicating whether the substring is present in each string.

3. `str.replace()`:

This method replaces occurrences of a substring with another substring in each string of a column.

```
import pandas as pd

# Create a DataFrame

data = {'Name': ['Alice Smith', 'Bob Johnson', 'Charlie Brown']}

df = pd.DataFrame(data)

# Replace 'Smith' with 'Johnson' in the 'Name' column
```

```
df['Name_replaced'] = df['Name'].str.replace('Smith', 'Johnson')
print(df)
```

Output:

```
            Name    Name_replaced
0     Alice Smith     Alice Johnson
1     Bob Johnson     Bob Johnson
2  Charlie Brown  Charlie Brown
```

In this example, the `str.replace()` method is used to replace occurrences of the substring 'Smith' with 'Johnson' in the 'Name' column. The result is assigned to a new column 'Name_replaced'.

4. `str.split()`:

This method splits each string in a column into a list of substrings based on a specified delimiter.

```
import pandas as pd
# Create a DataFrame
data = {'Name': ['Alice Smith', 'Bob Johnson', 'Charlie Brown']}
df = pd.DataFrame(data)
# Split the 'Name' column into first name and last name
df[['First Name', 'Last Name']] = df['Name'].str.split(' ', expand=True)
print(df)
```

Output:

```
            Name       First Name Last Name
0     Alice Smith      Alice        Smith
1     Bob Johnson        Bob       Johnson
2  Charlie Brown    Charlie       Brown
```

```

In this example, the `str.split()` method is used to split each string in the 'Name' column into a list of substrings using the space (' ') as the delimiter. The `expand=True` parameter ensures that the resulting substrings are assigned to separate columns 'First Name' and 'Last Name' using the `df[['First Name', 'Last Name']]` syntax.

## 23. Explain with example any two methods of creating Data Frame.

1. Creating a DataFrame from a Dictionary:

You can create a DataFrame by passing a dictionary to the `pd.DataFrame()` constructor. The keys of the dictionary represent column names, and the values represent the data in each column. Here's an example:

```
import pandas as pd

# Create a DataFrame from a dictionary
data = {'Name': ['Alice', 'Bob', 'Charlie'],
        'Age': [25, 30, 35],
        'City': ['New York', 'London', 'Paris']}
df = pd.DataFrame(data)
print(df)
```

Output:

```
      Name  Age      City
0    Alice   25  New York
1      Bob   30    London
2  Charlie   35     Paris
```

In this example, we pass the dictionary `data` to the `pd.DataFrame()` constructor. Each key in the dictionary represents a column name ('Name', 'Age',
```

and 'City'), and each value represents the data in that column. The resulting DataFrame `df` contains the specified columns and their respective data.

2. Creating a DataFrame from a CSV file:

You can also create a DataFrame by reading data from a CSV (Comma-Separated Values) file using the `pd.read_csv()` function. Here's an example:

```
import pandas as pd

# Read a CSV file and create a DataFrame
df = pd.read_csv('data.csv')
print(df)
```

Output:

```
    Name  Age     City
0   Alice   25  New York
1     Bob   30    London
2  Charlie   35     Paris
```

In this example, we use the `pd.read_csv()` function to read data from the CSV file named 'data.csv' and create a DataFrame. The CSV file should be in the same directory as the Python script or provide the correct file path to the function.

The resulting DataFrame `df` contains the data from the CSV file, and the column names are inferred from the header row of the CSV file.

These are two common methods of creating a DataFrame in Pandas. You can choose the most suitable approach based on your data source, whether it's a dictionary or an external file like CSV, Excel, or a database. Pandas provides various other methods for creating DataFrames, such as reading from Excel files, SQL databases, and more, offering flexibility in handling different types of data sources.

24. Explain Bar Graph creation using Matplot Library module.

To create a bar graph using the Matplotlib library in Python, you can use the `matplotlib.pyplot.bar()` function. The bar graph is a visual representation of categorical data with rectangular bars, where the length of each bar represents a specific value. Here's an example that demonstrates how to create a bar graph using Matplotlib:

```
import matplotlib.pyplot as plt

# Sample data

categories = ['A', 'B', 'C', 'D']

values = [10, 20, 15, 25]

# Create a bar graph

plt.bar(categories, values)

# Add labels and title

plt.xlabel('Categories')

plt.ylabel('Values')

plt.title('Bar Graph')

# Display the graph

plt.show()
```

In this example, we start by importing the `matplotlib.pyplot` module as `plt`. We define the sample data we want to plot, where `categories` represents the categories on the x-axis and `values` represents the corresponding values for each category.

Next, we create a bar graph using the `plt.bar()` function. We pass the `categories` and `values` as the first two arguments to the function.

To enhance the graph, we add labels and a title using the `plt.xlabel()`, `plt.ylabel()`, and `plt.title()` functions, respectively.

Finally, we display the graph using `plt.show()`.

Running this code will generate a bar graph with the categories on the x-axis and the corresponding values on the y-axis. Each category will be represented by a rectangular bar, and the length of each bar will represent its corresponding value.

You can customize the appearance of the bar graph further by adjusting the color, width, alignment, and other properties using additional parameters in the `plt.bar()` function. Matplotlib offers extensive customization options to create visually appealing and informative bar graphs.

25. Write a program to display histogram.

```
import matplotlib.pyplot as plt

# Sample data

data = [10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80]

# Create a histogram

plt.hist(data)

# Add labels and title

plt.xlabel('Values')

plt.ylabel('Frequency')

plt.title('Histogram')

# Display the histogram

plt.show()
```

In this example, we start by importing the `matplotlib.pyplot` module as `plt`. We define the sample data that we want to visualize using a histogram. The `data` list contains the values for which we want to create a histogram.

Next, we create a histogram using the `plt.hist()` function. We pass the `data` as the first argument to the function.

To enhance the histogram, we add labels and a title using the `plt.xlabel()`, `plt.ylabel()`, and `plt.title()` functions, respectively. These labels provide information about the x-axis, y-axis, and the title of the histogram.

Finally, we display the histogram using `plt.show()`.

Running this code will generate a histogram with bins representing the range of values and the frequency of occurrence for each bin. The x-axis represents the values, and the y-axis represents the frequency or count of values falling within each bin.

26. Write a Python program to display Pie Chart showing percentage of employees in each department. Assume there are 4 departments namely Sales , Production , HR and Finance.

```
import matplotlib.pyplot as plt

# Sample data

departments = ['Sales', 'Production', 'HR', 'Finance']

employee_counts = [40, 30, 20, 10]

# Create a pie chart

plt.pie(employee_counts, labels=departments, autopct='%1.1f%%')

# Add title

plt.title('Percentage of Employees in Each Department')

# Display the pie chart

plt.show()
```

In this example, we start by importing the `matplotlib.pyplot` module as `plt`. We define the sample data that we want to visualize. The `departments` list contains the department names, and the `employee_counts` list contains the corresponding number of employees in each department.

Next, we create a pie chart using the `plt.pie()` function. We pass the `employee_counts` as the first argument and the `departments` as the `labels` parameter to the function. The `autopct='%1.1f%%'` parameter is used to display the percentage values on each slice of the pie chart.

To enhance the pie chart, we add a title using the `plt.title()` function.

Finally, we display the pie chart using `plt.show()`.

Running this code will generate a pie chart where each department is represented by a slice. The size of each slice represents the percentage of employees in that department. The legend labels indicate the department names, and the percentage values are displayed on each slice.

You can further customize the appearance of the pie chart by adjusting parameters such as colors, explode (to highlight a particular slice), shadow, and more using additional arguments in the `plt.pie()` function. Matplotlib provides various options to create visually appealing and informative pie charts.

27. Write a Python Program to create Line Graph showing number of students of a college in various Years. Consider 8 years data.

```
import matplotlib.pyplot as plt

# Sample data

years = [2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022]

student_counts = [500, 600, 700, 800, 900, 1000, 1100, 1200]

# Create a line graph

plt.plot(years, student_counts, marker='o')

# Add labels and title

plt.xlabel('Year')

plt.ylabel('Number of Students')

plt.title('Number of Students in College Over Years')

# Display the line graph

plt.show()
```

In this example, we start by importing the `matplotlib.pyplot` module as `plt`. We define the sample data that represents the years and the corresponding number of students in the college. The `years` list contains the years, and the `student_counts` list contains the respective number of students in each year.

Next, we create a line graph using the `plt.plot()` function. We pass the `years` as the first argument and `student_counts` as the second argument to the function. The `marker='o'` parameter is used to display circular markers at each data point.

To enhance the line graph, we add labels and a title using the `plt.xlabel()`, `plt.ylabel()`, and `plt.title()` functions, respectively.

Finally, we display the line graph using `plt.show()`.

Running this code will generate a line graph where the x-axis represents the years, and the y-axis represents the number of students in the college. Each data point is connected by a line, and circular markers are displayed at each data point.

You can further customize the appearance of the line graph by adjusting parameters such as line color, line style, marker type, and more using additional arguments in the `plt.plot()` function. Matplotlib offers various options to create visually appealing and informative line graphs.